

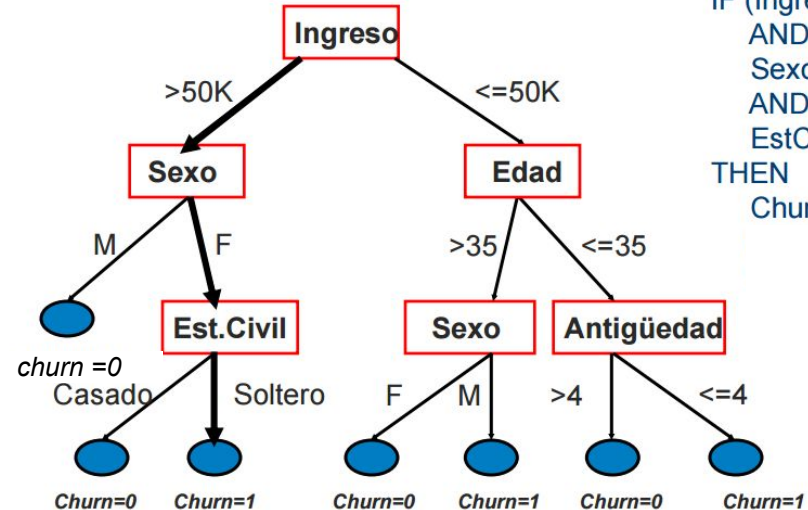
Unidad 5

Definición formal y tipos de árboles. Estructura de un árbol de decisión. Criterios de división para árboles de decisión para regresión y clasificación. Entropía, impureza de Gini, ganancia de información. Tuning de hiperparámetros en árboles de decisión. ID3, C4.5 y CART. Aplicaciones espaciales.

Generalidades

Definición formal

Un árbol de decisión es un algoritmo de aprendizaje supervisado no paramétrico, lo que significa que no realiza suposiciones sobre la distribución de los datos, dándole una notable flexibilidad para modelar relaciones complejas. Se utiliza tanto para tareas de clasificación como de regresión.



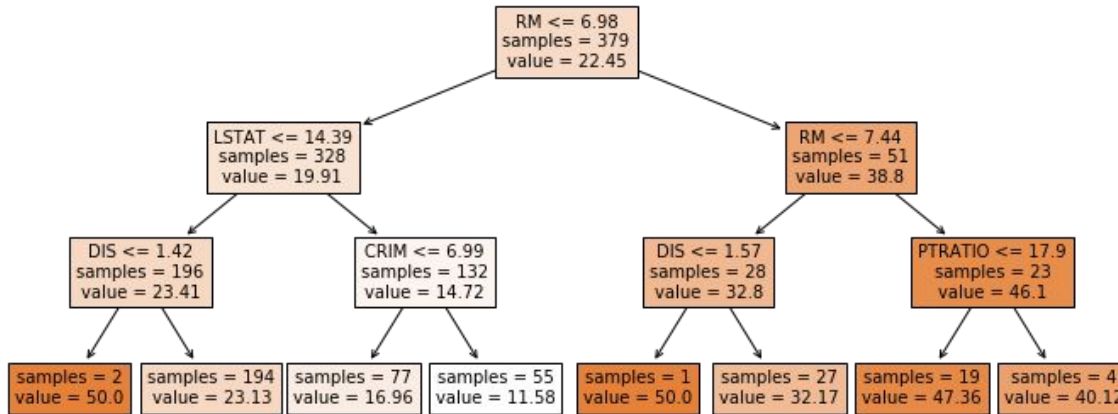
RULE:

IF (Ingreso >50K
AND
Sexo = F
AND
EstCivil = Soltero)
THEN
Churn = 1

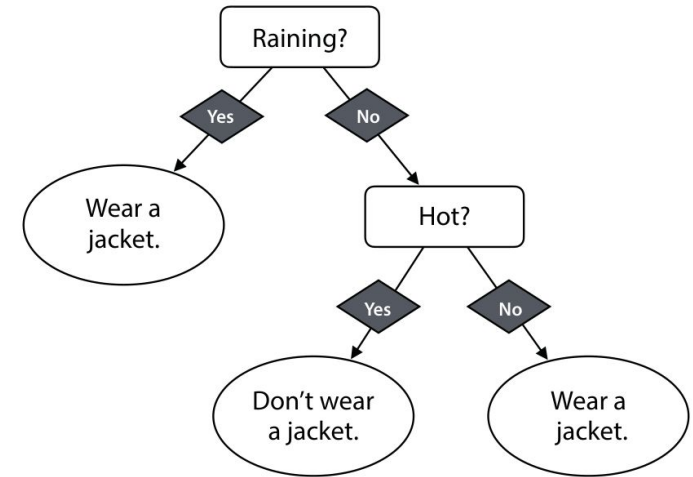
Árbol de decisión para clasificación
con variables cuantitativas y
cualitativas

Tipos de árboles

Árbol de decisión para regresión



Árbol de decisión para clasificación

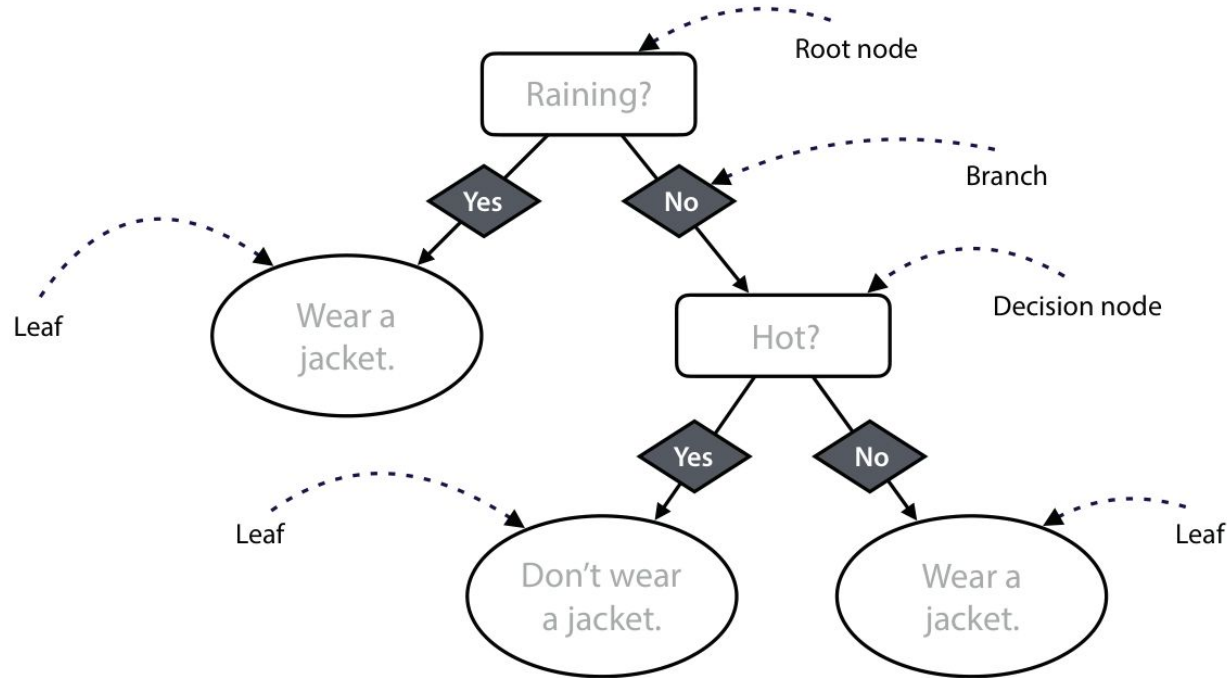


Estructura de un árbol de decisión

La estructura de un árbol de decisión es jerárquica y se asemeja a un diagrama de flujo. Este diseño descompone un problema complejo en una serie de pasos simples y estructurados, donde cada decisión conduce a una nueva ramificación hasta alcanzar un resultado final. Sus componentes son:

- Nodo raíz (Root node): punto de partida del árbol y representa el conjunto de datos completo antes de cualquier división. De él parten todas las ramas iniciales.
- Nodos internos o de decisión (internal / decision nodes): (cada uno de estos nodos representa una “pregunta” o una prueba sobre una característica específica del conjunto de datos. El resultado de esta prueba determina que camino a seguir.
- Ramas (branches): son las líneas que conectan los nodos. Cada rama representa una posible respuesta a la pregunta del nodo anterior (“sí”, “no”) y muestra la relación entre las variables.
- Nodos u hojas terminales (leaf/terminal nodes): son los puntos finales del árbol, donde no se realizan más divisiones. Cada nodo hoja representa una decisión final: una etiqueta de clase o un valor numérico.

Estructura de los árboles de decisión



Enfoque “divide y vencerás”

La construcción de un árbol de decisión se basa en un algoritmo de segmentación recursiva. Este proceso sigue una estrategia “divide y vencerás”: comienza con un conjunto de datos completo en el nodo raíz y lo divide sucesivamente en subconjuntos más pequeños y homogéneos. Este procedimiento se repite en cada subconjunto hasta que se cumple un criterio de parada, como que todos los datos en un nodo pertenezcan a la misma clase.

Este enfoque es inherentemente “greedy” (codicioso). En cada nodo, el algoritmo busca la mejor división posible en ese momento seleccionando la característica y el umbral que optimizan el criterio de división localmente, sin considerar si esta decisión conducirá al mejor árbol a nivel global.

Criterios de división para árboles de clasificación

El objetivo de un criterio de división en clasificación es medir la pureza de un nodo, es decir, cuán homogéneo es con respecto a las clases de la variable objetivo. Los dos criterios más comunes son:

- Impureza de Gini (Gini impurity): mide la probabilidad de clasificar incorrectamente una muestra si se etiquetara al azar según la distribución de clases en el nodo. Un índice de Gini de 0 indica pureza total (todas las muestras son de una sola clase). El algoritmo elige la división que resulta en el menor índice de Gini promedio ponderado en los nodos hijos.

$$Gini(t) = 1 - \sum_{c \in C} p_{t,c}^2$$

- La Ganancia de Información es la reducción de la entropía lograda al dividir un nodo. El algoritmo selecciona la característica que maximiza esta ganancia, eligiendo así la división más informativa.

$$Entropía = - \sum_k p_k \log_2 p_k$$

Criterios de división para árboles de regresión

Cuando la variable objetivo es continua, el criterio de división cambia. En lugar de buscar la pureza de clase, el objetivo es crear nodos donde los valores de la variable objetivo sean lo más similares posibles.

- Reducción de la Varianza: el algoritmo elige la división que minimiza la suma ponderada de las varianzas de los nodos hijos. Esto es equivalente a minimizar el error cuadrático medio (MSE) con respecto a la media de cada nodo. La división con la menor varianza ponderada es la seleccionada.

En clasificación, nuestro objetivo es que un nodo permita la separación de los datos en grupos los más homogéneos posibles.

En regresión, sin embargo, nos interesa que los nodos permitan separar los datos en grupos en los cuales el error de predicción es bajo.

Cómo dividimos un árbol de decisión

Attributes				Classes
Outlook	Temperature	Humidity	Windy	Play Golf
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Overcast	Cool	Normal	TRUE	Yes
Rainy	Mild	High	FALSE	No
Rainy	Cool	Normal	FALSE	Yes
Sunny	Mild	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Sunny	Mild	High	TRUE	No

Figure 1: Exercise on Decision Trees

Este conjunto de datos es un **ejemplo de minería de datos y aprendizaje automático** que se utiliza específicamente para ilustrar la construcción de un **Árbol de Decisión** para una tarea de **Clasificación**.

El objetivo del dataset es predecir si una persona **"Jugará Golf"** (Play Golf) basándose en las condiciones climáticas.

El dataset contiene **14 observaciones** (filas) y permite construir reglas sencillas (como las ramas de un árbol de decisión) para clasificar si se debe jugar al golf o no, dadas las condiciones del clima.

Cómo dividimos un árbol de decisión

Outlook	Temperature	Humidity	Windy	Play Golf
Sunny	Mild	Normal	FALSE	Yes
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Sunny	Mild	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Overcast	Cool	Normal	TRUE	Yes
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Rainy	Mild	High	FALSE	No
Rainy	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes

Table 7: Initial Split using Outlook

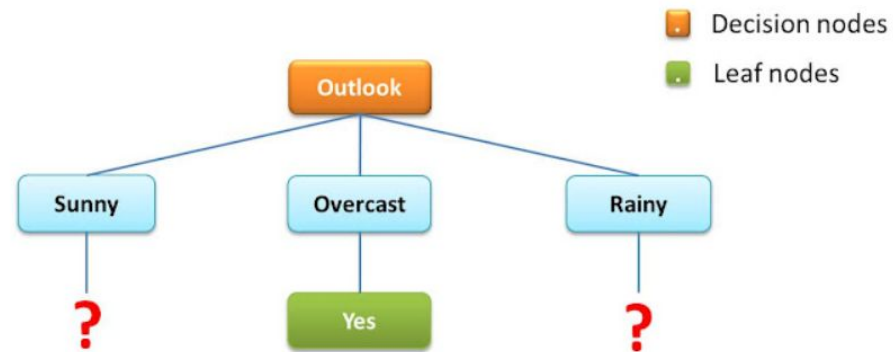


Figure 2: Decision Tree after first split

Cómo dividimos un árbol de decisión

Outlook	Temperature	Humidity	Windy	Play Golf
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Rainy	Mild	High	FALSE	No

Rainy	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes

Table 8: Split using Humidity

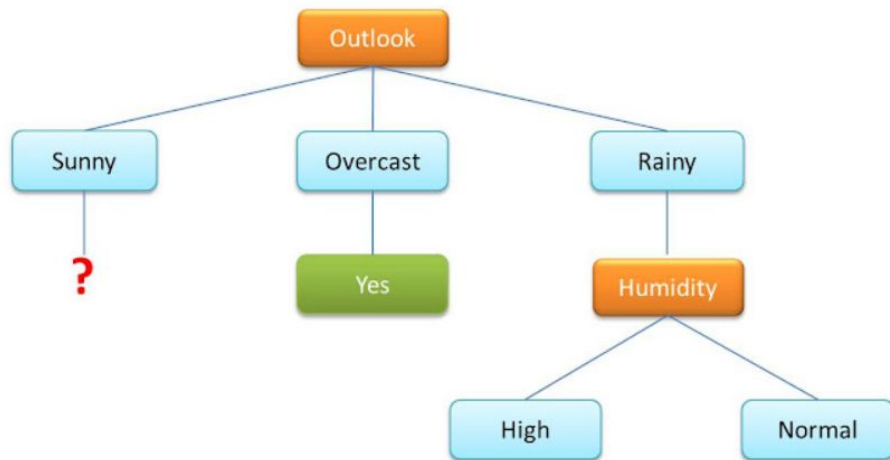


Figure 3: Split using the Humidity Attribute

Cómo dividimos un árbol de decisión

Outlook	Temperature	Humidity	Windy	Play Golf
Sunny	Mild	Normal	FALSE	Yes
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes

Sunny	Cool	Normal	TRUE	No
Sunny	Mild	High	TRUE	No

Table 9: Split using Windy Attribute

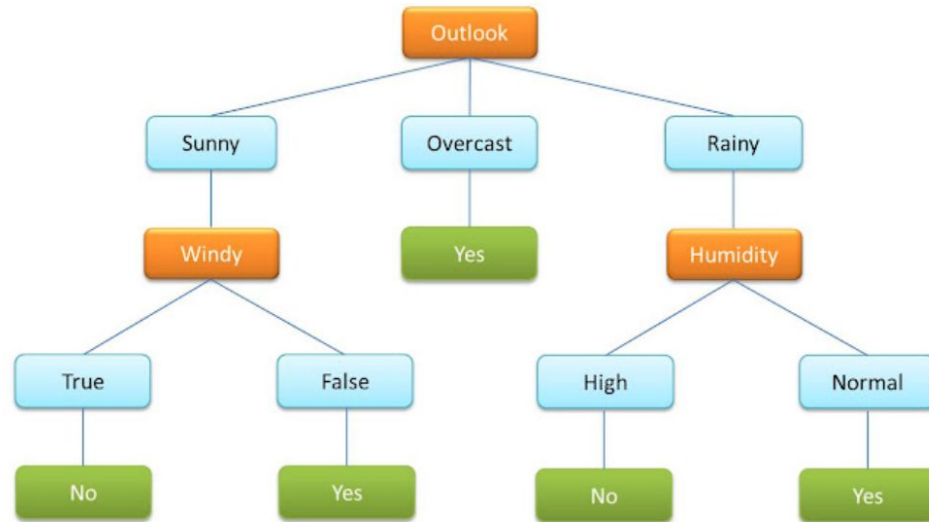


Figure 4: Final Decision Tree

Árboles de decisión para regresión

Límite de decisión de un árbol

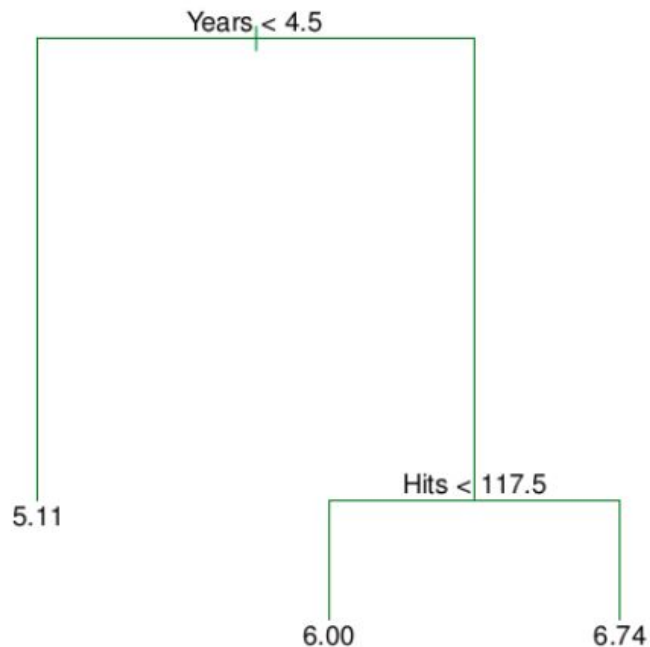
Veamos cómo son los límites de decisión que crea un árbol de decisión.

Para ello, vamos a ver un ejemplo de la librería ISLR, sobre jugadores de béisbol.

```
library(ISLR)
library(magrittr)
library(tidyverse)
data(Hitters)
head(Hitters) %>%
  select(Years, Salary, Hits)
```

```
##           Years Salary Hits
## -Andy Allanson      1     NA   66
## -Alan Ashby        14  475.0   81
## -Alvin Davis        3  480.0  130
## -Andre Dawson       11  500.0  141
## -Andres Galarraga    2   91.5   87
## -Alfredo Griffin    11  750.0  169
```

Límite de decisión de un árbol



```
Hitters %>%  
  select(Years, Salary, Hits) %>%  
  summary()
```

##	Years	Salary	Hits
##	Min. : 1.000	Min. : 67.5	Min. : 1
##	1st Qu.: 4.000	1st Qu.: 190.0	1st Qu.: 64
##	Median : 6.000	Median : 425.0	Median : 96
##	Mean : 7.444	Mean : 535.9	Mean : 101
##	3rd Qu.: 11.000	3rd Qu.: 750.0	3rd Qu.: 137
##	Max. : 24.000	Max. : 2460.0	Max. : 238
##		NA's : 59	

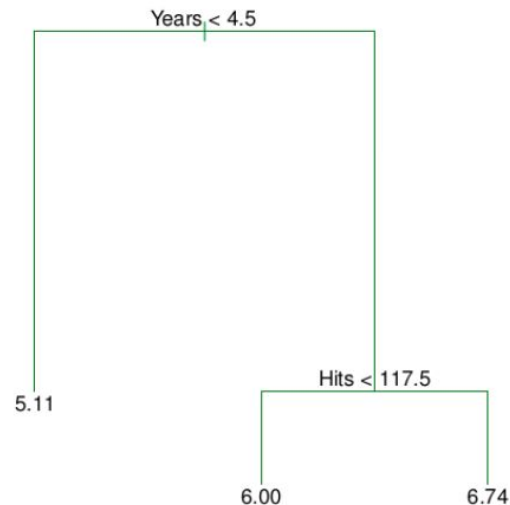
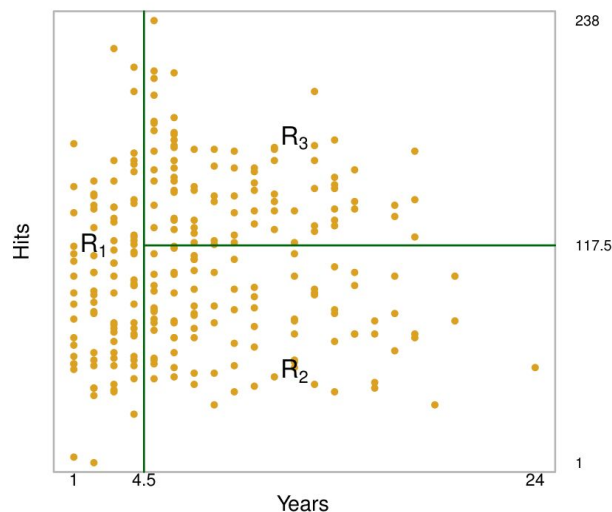
Límite de decisión de un árbol

$R_1 = \{X | Year < 4.5\}$: jugadores que han jugado menos de 4.5 años.

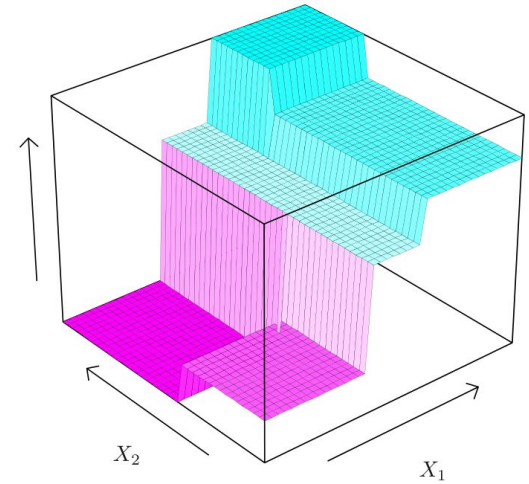
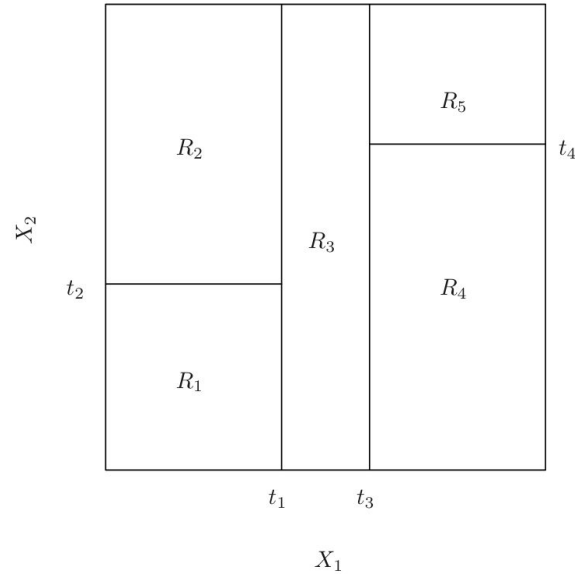
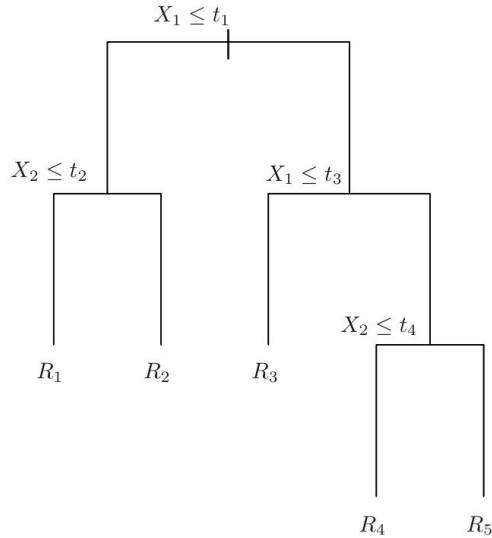
$R_2 = \{X | Year \geq 4.5, Hits < 117.5\}$: jugadores que han jugado 4.5 años o más y que han conseguido menos de 117.5 bateos.

$R_3 = \{X | Year \geq 4.5, Hits \geq 117.5\}$: jugadores que han jugado 4.5 años o más y que han conseguido 117.5 o más bateos.

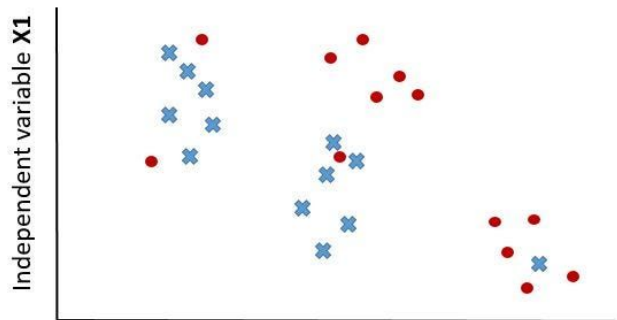
El valor que aparece en cada hoja es la media de las observaciones que están en esa hoja.



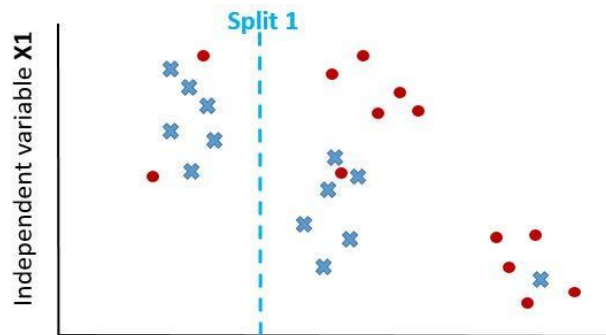
Límite de decisión de un árbol



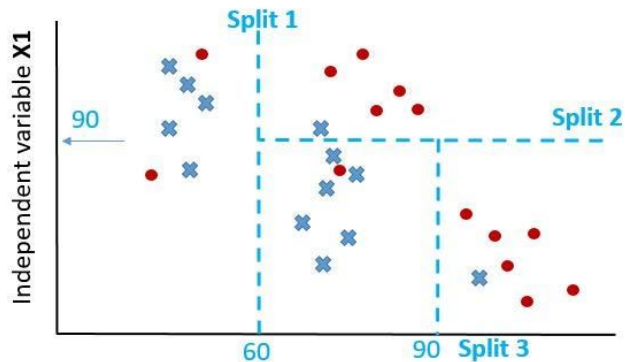
Límite de decisión de un árbol



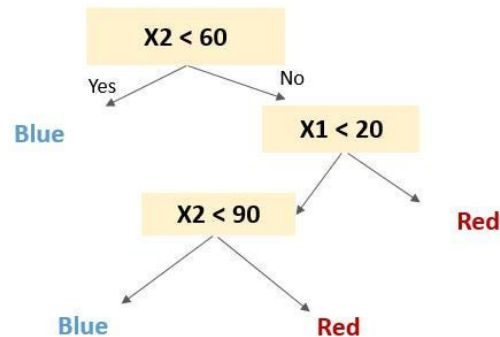
Independent variable X2



Independent variable X2



Independent variable X2



Example 14.2 Growing a maximum-depth regression tree

The recursive procedure for growing a deep regression tree is illustrated in [Figure 14.7](#). We begin (on the left) by fitting a stump to the original dataset. As we move from left to right the recursion proceeds, with each leaf of the preceding tree split in order to create the next, deeper tree. As can be seen in the rightmost panel, a tree with maximum depth of four is capable of representing the training data perfectly.

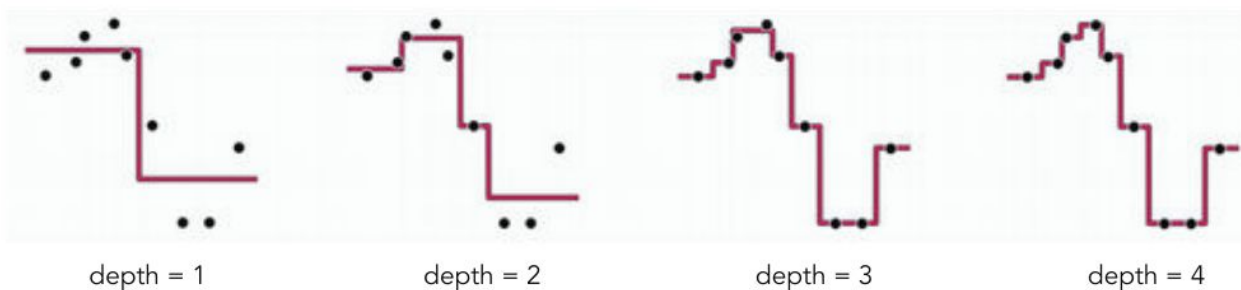
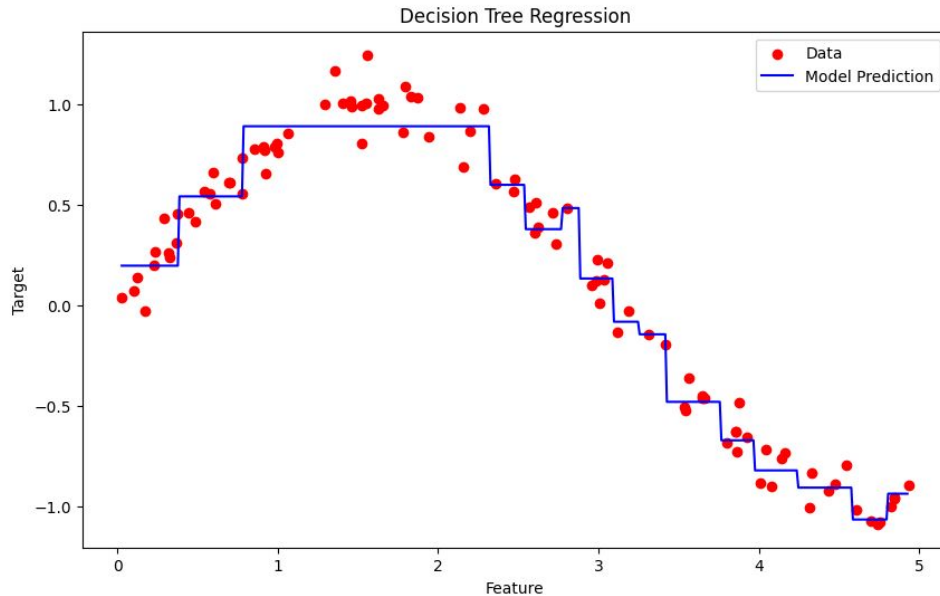


Figure 14.7 Figure associated with [Example 14.2](#). See text for details.

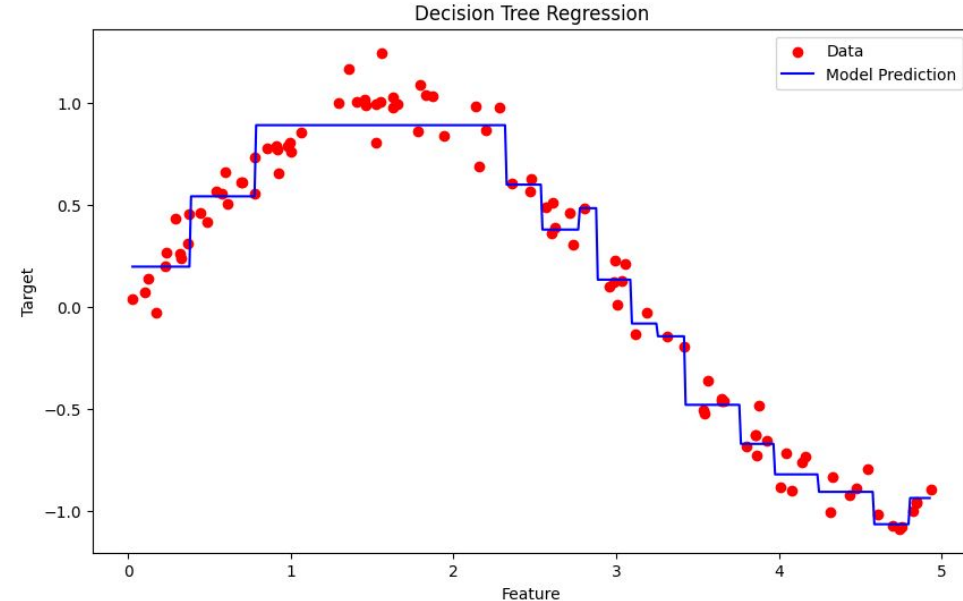
Árbol de decisión para regresión



El algoritmo del Árbol de Regresión no predice una línea curva, sino que divide el rango de la característica de entrada (eje X) en varias **regiones rectangulares disjuntas**. En este gráfico unidimensional, estas regiones son simplemente **intervalos** en el eje X. Por cada una de estas regiones (intervalos) creadas, el árbol hace una única y simple predicción: el **valor promedio** de la variable objetivo (**Target**) para todos los puntos de datos (puntos rojos) que caen dentro de ese intervalo.

Cada línea azul horizontal representa este **valor promedio** constante calculado para una región R_m específica.

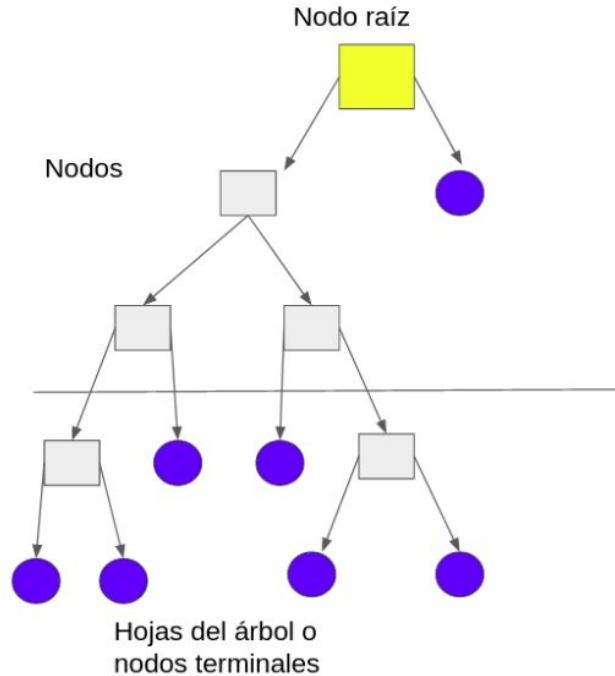
Árbol de decisión para regresión



Los **saltos verticales** en la línea azul ocurren exactamente en los **puntos de división** o **nodos internos** que el algoritmo del árbol encontró óptimos. Estos puntos de división son elegidos para maximizar la **reducción de la varianza** (o el error cuadrático medio) dentro de cada nueva región.

Cuanto **más complejo** (más profundo) es el árbol, más regiones crea y más **pequeños** se vuelven los segmentos horizontales, tratando de ajustarse más a cada punto de datos, como se ve en el gráfico. Si el árbol fuera muy simple (poca profundidad), veríamos solo unos pocos y muy largos escalones.

Overfitting en árboles de decisión

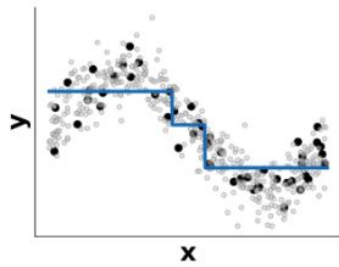


Menos overfitting:
mas sesgo, menos varianza

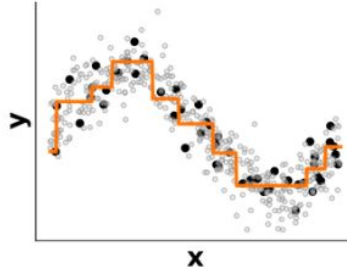
Por acá estaría la profundidad ideal
del árbol

Más overfitting:
menos sesgo, más varianza.

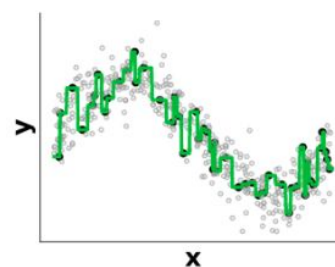
Overfitting en árboles de regresión



Underfitting



Mejor balance



Overfitting

¿Cómo controlo el overfitting en árboles?

- Controlar profundidad del árbol.
- Controlar la cantidad de hojas (para armar una cantidad fija de grupos de datos).
- Controlar la mínima cantidad de datos que están contenidos dentro de cada hoja (para hacer, por ejemplo, promedios más robustos).
- Controlar el hiperparámetro de regularización.

- Los árboles son fáciles de interpretar aun cuando las relaciones entre predictores son complejas.
- Los modelos basados en un solo árbol (no es el caso de Random Forest y Boosting) se pueden representar gráficamente aun cuando el número de predictores es mayor de 3.
- Los árboles pueden, en teoría, manejar tanto predictores numéricos como categóricos sin tener que crear variables dummy o one-hot-encoding. En la práctica, esto depende de la implementación del algoritmo que tenga cada librería.
- Al tratarse de métodos no paramétricos, no es necesario que se cumpla ningún tipo de distribución específica.
- Por lo general, requieren mucha menos limpieza y preprocesado de los datos en comparación con otros métodos de aprendizaje estadístico (por ejemplo, no requieren estandarización).

- No se ven muy influenciados por outliers.
- Si para alguna observación, el valor de un predictor no está disponible, a pesar de no poder llegar a ningún nodo terminal, se puede conseguir una predicción empleando todas las observaciones que pertenecen al último nodo alcanzado. La precisión de la predicción se verá reducida pero al menos podrá obtenerse.
- Son muy útiles en la exploración de datos, permiten identificar de forma rápida y eficiente las variables (predictores) más importantes.
- Son capaces de seleccionar predictores de forma automática.
- Pueden aplicarse a problemas de regresión y clasificación.

- La capacidad predictiva de los modelos basados en un único árbol es bastante inferior a la conseguida con otros modelos. Esto es debido a su tendencia al **overfitting** y alta varianza. Sin embargo, existen técnicas más complejas que, haciendo uso de la combinación de múltiples árboles (**bagging, random forest, boosting**), consiguen mejorar en gran medida este problema.
- Son sensibles a datos de entrenamiento desbalanceados (una de las clases domina sobre las demás).
- Cuando tratan con predictores continuos, pierden parte de su información al categorizarlos en el momento de la división de los nodos.
- Tal y como se describe más adelante, la creación de las ramificaciones de los árboles se consigue mediante el algoritmo de **recursive binary splitting**. Este algoritmo identifica y evalúa las posibles divisiones de cada predictor acorde a una determinada medida (**RSS, Gini, entropía...**). Los predictores continuos tienen mayor probabilidad de contener, solo por azar, algún punto de corte óptimo, por lo que suelen verse favorecidos en la creación de los árboles.
- No son capaces de extrapolar fuera del rango de los predictores observado en los datos de entrenamiento.

ID3 - Iterative Dichotomizer 3

Descrito por Quinlan, J. R. (1986). Induction of decision trees. Machine learning, 1 (1), 81-106. (Es uno de los primeros algoritmos de árboles de decisión).

Maximiza la ganancia de información / minimiza la entropía (Criterio de *split* o división).

Maneja exclusivamente *features* (atributos) discretos, binarios y multi-categoricos.
(No puede manejar *features* numéricos/continuos directamente).

Utiliza *splits* multi-categoricos (Puede crear ramas para cada valor único de un atributo).

Tiende a generar árboles pequeños y amplios (es decir, con pocas profundidades y muchas divisiones tempranas, a diferencia de algoritmos como CART, que pueden ser más profundos).

Algoritmo C4.5 (Evolución de ID3).

Descrito por Quinlan, J. R. (1993). **C4. 5: Programming for machine learning.** Morgan Kauffmann, 38, 48.

Maneja *features* (atributos) continuos y discretos. (Los atributos continuos se convierten en *splits* binarios al encontrar un punto de corte óptimo).

Criterio de *split* (división): Gain Ratio. (Se utiliza para mitigar el sesgo de *Information Gain* (usado en ID3) hacia atributos con muchos valores únicos).

Puede manejar atributos (datos) faltantes (ignora los valores nulos al calcular el *Gain Ratio*).

Adición Clave: Incluye una fase de **poda (*pruning*)** por defecto para simplificar el árbol y reducir el **sobreajuste (*overfitting*)**.

Algoritmo CART (Classification and Regression Trees).

Descrito por Breiman, L. (1984). Classification and regression trees. Belmont, Calif: Wadsworth International Group.

Versatilidad: Es capaz de generar tanto **árboles de clasificación** como **árboles de regresión** (de ahí su nombre). Para ello utiliza la **impureza de Gini** como criterio de *split* en los árboles de clasificación y utiliza la **reducción de varianza** (o error cuadrático medio) como criterio de *split* en los árboles de regresión.

Maneja *features* continuos y discretos.

Estructura: Trabaja **estrictamente con *splits* binarios** (dos ramas por nodo), independientemente del número de categorías del *feature*. Esto tiende a producir árboles **más altos y estrechos** que ID3 o C4.5.

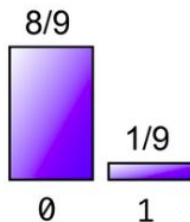
Ventaja/Desventaja: Los *splits* binarios a menudo generan árboles más grandes que C4.5, lo que podría hacerlos más difíciles de interpretar, pero son **fáciles de implementar** y pueden tener un buen rendimiento.

Adición Clave: A diferencia de ID3, CART utiliza la **poda de coste-complejidad (*cost-complexity pruning*)** para manejar el sobreajuste.

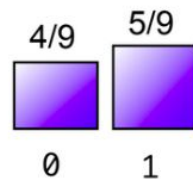
Árboles de decisión para clasificación

La **Entropía** (también conocida como Entropía de Shannon) mide la **cantidad promedio de información** necesaria para clasificar correctamente una muestra dentro de un nodo. En otras palabras, cuantifica el **desorden** o la **mezcla de clases** en un conjunto de datos.

$$Entropia(S) = \sum_{i=1}^n -P_i \log_2 P_i \quad (1)$$



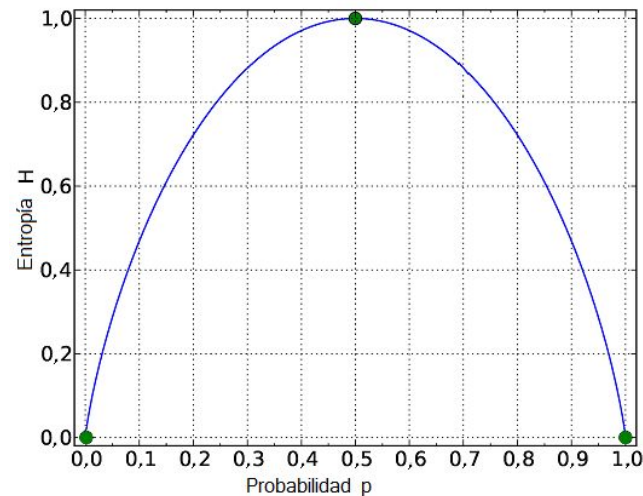
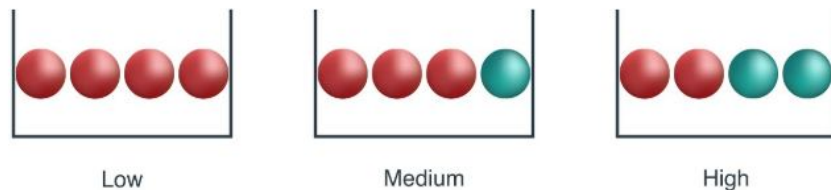
$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$



$$-\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9} \approx 0.99$$

Interpretación de la entropía

- **Entropía = 0 (Pureza Total):** Significa que el nodo es **puro**. Todas las muestras pertenecen a la **misma clase**. No hay incertidumbre.
- **Entropía = 1 (Impureza Máxima para 2 Clases):** Significa que el nodo está perfectamente **mezclado**. Para un problema binario, esto ocurre cuando hay una división 50/50 entre las dos clases, lo que resulta en la máxima incertidumbre.

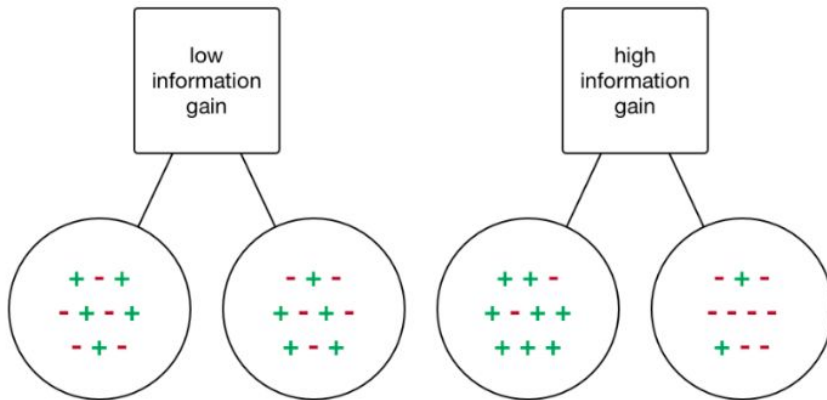


La entropía es 0 cuando la probabilidad es 0 y 1, y es máxima cuando la probabilidad es 0.5

Ganancia de Información (Information Gain)

La ganancia de información es una propiedad estadística que mide qué tan bien un atributo determinado separa los ejemplos de entrenamiento de acuerdo con su clasificación de destino. La construcción de un árbol de decisiones consiste en encontrar un atributo que devuelva la **mayor ganancia de información** y la **menor entropía**.

$$IG = Entropy(parent) - weighted\ average[Entropy(children)]$$



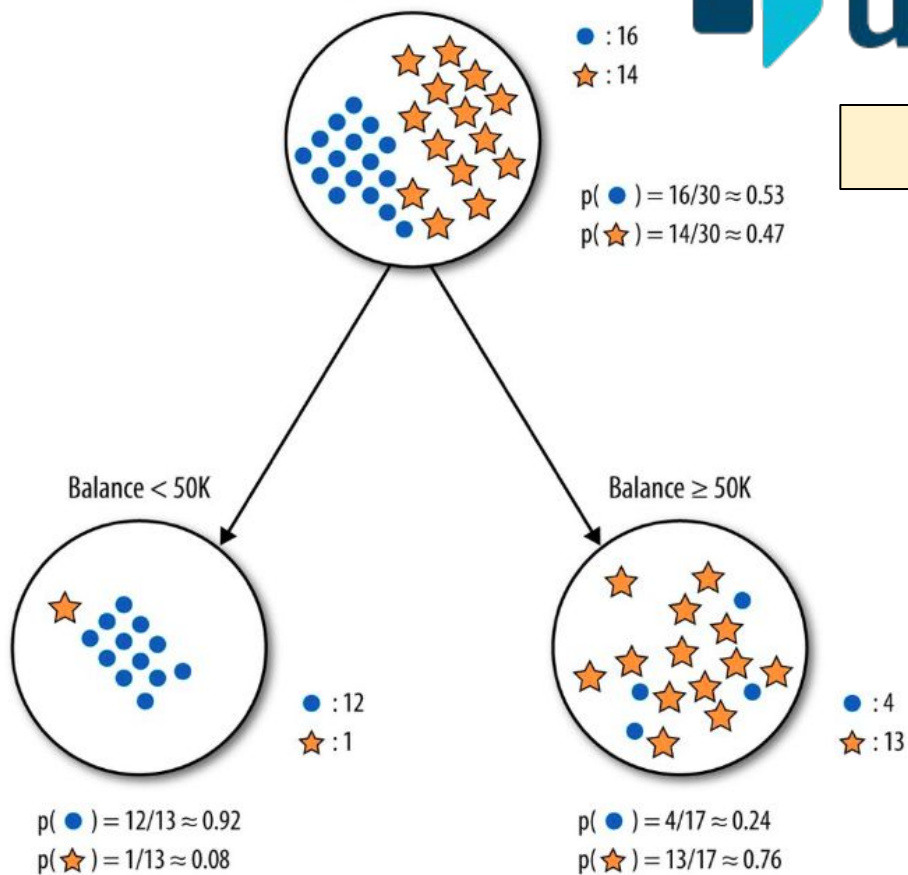
Veamos cómo funciona un árbol de decisión

Supongamos que tenemos que elegir si entregar o no un préstamo, y tenemos dos variables para decidir si lo hacemos:

- Balance: saldo de cuenta
- Residence: lugar de residencia.

Opción 1: Dividir el nodo raíz según Balance

Entire population (30 instances)



n=30

n=13

n=17

$$E(\text{Parent}) = -\frac{16}{30}\log_2\left(\frac{16}{30}\right) - \frac{14}{30}\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(\text{Balance} < 50K) = -\frac{12}{13}\log_2\left(\frac{12}{13}\right) - \frac{1}{13}\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(\text{Balance} > 50K) = -\frac{4}{17}\log_2\left(\frac{4}{17}\right) - \frac{13}{17}\log_2\left(\frac{13}{17}\right) \approx 0.79$$

Cálculo la entropía
entre nodo padre e
hijo

Weighted Average of entropy for each node:

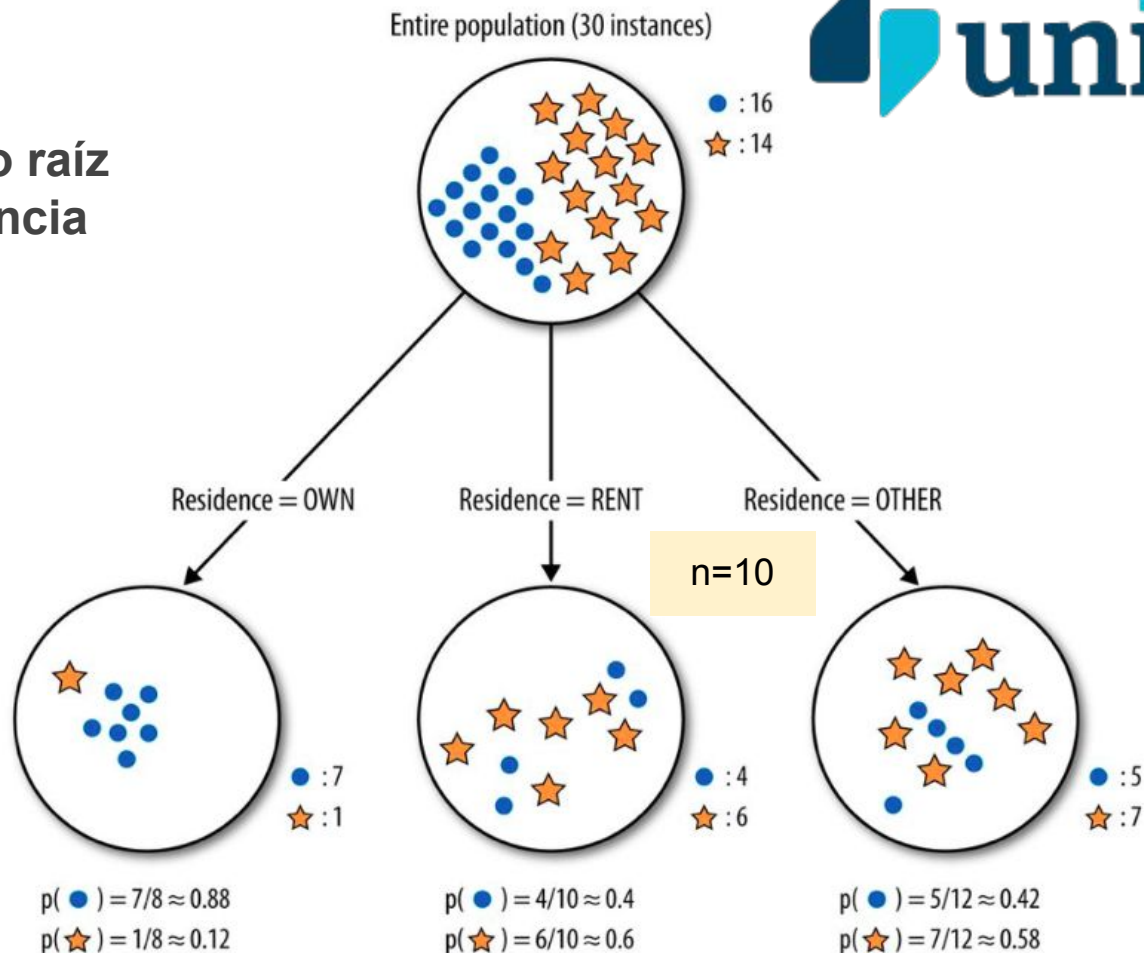
$$\begin{aligned} E(\text{Balance}) &= \frac{13}{30} \times 0.39 + \frac{17}{30} \times 0.79 \\ &= 0.62 \end{aligned}$$

Information Gain:

$$\begin{aligned} IG(\text{Parent}, \text{Balance}) &= E(\text{Parent}) - E(\text{Balance}) \\ &= 0.99 - 0.62 \\ &= 0.37 \end{aligned}$$

Cálculo de la
Ganancia de
Información

Opción 2: Dividir el nodo raíz según Residencia



$$E(\text{Residence} = \text{OWN}) = -\frac{7}{8}\log_2\left(\frac{7}{8}\right) - \frac{1}{8}\log_2\left(\frac{1}{8}\right) \approx 0.54$$

$$E(\text{Residence} = \text{RENT}) = -\frac{4}{10}\log_2\left(\frac{4}{10}\right) - \frac{6}{10}\log_2\left(\frac{6}{10}\right) \approx 0.97$$

$$E(\text{Residence} = \text{OTHER}) = -\frac{5}{12}\log_2\left(\frac{5}{12}\right) - \frac{7}{12}\log_2\left(\frac{7}{12}\right) \approx 0.98$$

Cálculo la entropía de los nodos hijos

Weighted Average of entropies for each node:

$$E(\text{Residence}) = \frac{8}{30} \times 0.54 + \frac{10}{30} \times 0.97 + \frac{12}{30} \times 0.98 = 0.86$$

Information Gain:

$$\begin{aligned} IG(\text{Parent}, \text{Residence}) &= E(\text{Parent}) - E(\text{Residence}) \\ &= 0.99 - 0.86 \\ &= 0.13 \end{aligned}$$

Cálculo de la Ganancia de Información

Cuanto más homogéneas son las ramas del árbol que se genera al seleccionar el algoritmo una variable

- **hay mayor ganancia de información**
- **hay menos entropía**

Esto se traduce en un mejor modelo para la predicción que deseamos realizar.

Relación de Ganancia (Gain Ratio)

El **Gain Ratio** se define como el **cociente** entre la **Ganancia de Información (Information Gain)** y la **Entropía del nodo raíz (root node)**.

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{SplitInfo}} = \frac{\text{Entropy (before)} - \sum_{j=1}^K \text{Entropy}(j, \text{after})}{\sum_{j=1}^K w_j \log_2 w_j}$$

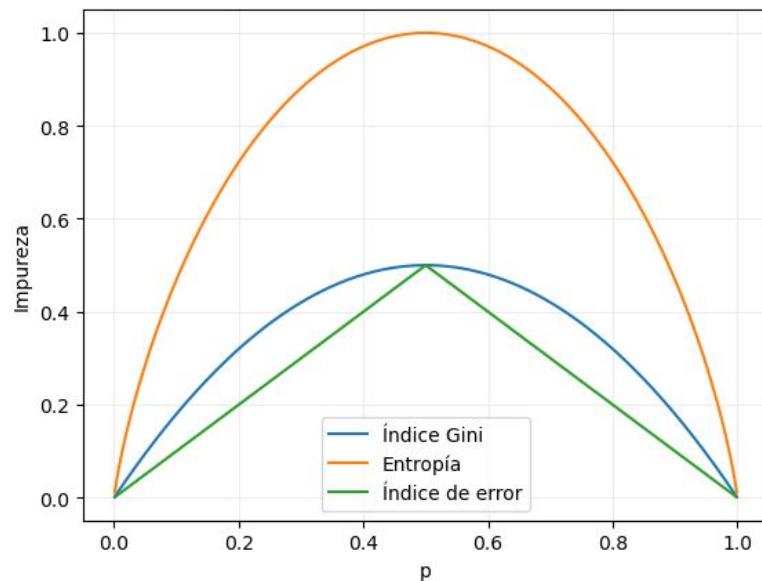
El principal objetivo de usar el Gain Ratio es **corregir el sesgo** del algoritmo ID3 (que solo usa *Information Gain*).

Utilizado en C4.5

- Medida de impureza en CART como contraposición a la entropía.

$$Gini(t) = 1 - \sum_{i=1}^j P(i|t)^2$$

- En la práctica no importa si utilizamos la entropía o la impureza de Gini, porque ambos criterios tienen forma de campana.
- Es más fácil computar el Gini, lo que hace el cálculo sea más eficiente.



Búsqueda de Hiperparámetros

- **criterion:** Es la función que mide la calidad de una división.

Para clasificación admite: "gini" (impureza Gini), "entropy" y "log_loss" (ambas para la ganancia de información de Shannon).

Para regresión admite: "squared_error" (error cuadrático medio), "friedman_mse" (error cuadrático medio con la mejora de Friedman), "absolute_error" (error absoluto medio) y "poisson" (desvianza de Poisson)

- **splitter:** La estrategia que se utiliza para elegir la división en cada nodo. Puede ser "best" (la mejor división) o "random" (la mejor división aleatoria).
- **max_depth:** La profundidad máxima del árbol. Si se define como nulo (None), los nodos se seguirán expandiendo hasta que todas las hojas sean puras o hasta que contengan menos de `min_samples_split` muestras.
- **min_samples_split:** El número o fracción mínima de muestras que se requieren para poder dividir un nodo interno.

- **min_samples_leaf**: El número o fracción mínima de muestras que deben quedar en un nodo hoja tras una división. Una división solo se considerará si deja al menos esta cantidad de muestras en las ramas izquierda y derecha, lo cual puede ayudar a suavizar el modelo.
- **min_weight_fraction_leaf**: La fracción mínima ponderada del total de pesos de todas las muestras de entrada requerida para que un nodo sea una hoja.
- **max_features**: El número de características (o una fracción de ellas, la raíz cuadrada "`sqrt`", o el logaritmo en base dos "`log2`") que el algoritmo considerará al buscar la mejor división.
- **random_state**: Controla la aleatoriedad del estimador, como la permutación de características en cada división. Fijar este valor con un número entero garantiza un comportamiento determinista al ajustar el modelo

- **max_leaf_nodes**: Define un límite máximo para la cantidad de nodos hoja. Si se especifica, el árbol crecerá de forma que priorice los mejores nodos definidos por la reducción relativa de la impureza.
- **min_impurity_decrease**: Un nodo se dividirá únicamente si dicha división produce una disminución de la impureza que sea mayor o igual a este valor.
- **ccp_alpha**: Es un parámetro de complejidad utilizado para el proceso de poda (Minimal Cost-Complexity Pruning). El algoritmo elegirá el subárbol con la mayor complejidad de costo que sea menor a este valor de alfa.
- **class_weight** (*Exclusivo de clasificación*): Permite asignar pesos a las diferentes clases mediante un diccionario (`{class_label: weight}`). También incluye el modo "balanced", el cual ajusta automáticamente los pesos de forma inversamente proporcional a la frecuencia de cada clase en los datos.

Hiperparámetros en árboles de decisión

- Documentación de árboles de decisión de clasificación:
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- Documentación de árboles de decisión de regresión:
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html>



1. ¿Cómo realizar una división (*split*)?

Esto se refiere al **Criterio de Selección de Atributos**:

- **Criterio de *Split* (Bondad de Ajuste)**: Determinar la **métrica de impureza** a usar para seleccionar el mejor atributo y el mejor umbral de división. Ejemplos incluyen:
 - **Ganancia de Información** (*Information Gain*), **Ratio de Ganancia** (*Gain Ratio*) o **Impureza de Gini** (*Gini Impurity*).
- **Tipo de *Split***: Elegir si la división será:
 - ***Split* Binario (Dos ramas)**: Típico de CART.
 - ***Split* Multi-Catégorico (Una rama por valor)**: Típico de ID3 y C4.5.

2. ¿Cuándo detener el crecimiento del árbol? (Criterios de Parada)

Esto define cuándo un nodo se convierte en una **hoja** (nodo terminal):

- **Pura Homogeneidad:** Si un nodo contiene muestras que **pertenecen exclusivamente a la misma clase** (el nodo es "puro"). No es necesario dividir más.
- **Falta de Atributos Útiles (o Indistinguibilidad):** Si **todos los valores de los *features* restantes son idénticos** para las muestras en ese nodo, no se puede realizar una división adicional que aporte valor.
- **Adición Clave (Hiperparámetros):** El crecimiento también se detiene basándose en **restricciones predefinidas** para prevenir el sobreajuste (*overfitting*). Ejemplos:
 - Alcanzar la **profundidad máxima** del árbol (e.g., Profundidad=10).
 - Alcanzar el **número mínimo de muestras** requeridas para una división (e.g., Mínimo de 20 muestras para dividirse).
 - Alcanzar el **número mínimo de muestras** en un nodo hoja (e.g., Mínimo de 5 muestras por nodo hoja).

Aplicaciones Espaciales

Focal-Test-Based Spatial Decision Tree Learning: A Summary of Results



(a) Features RGB 2008



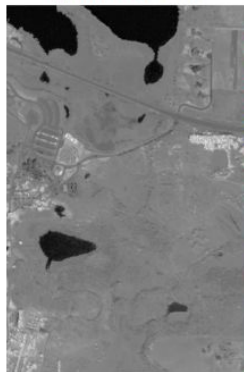
(b) Feature NDVI 2008



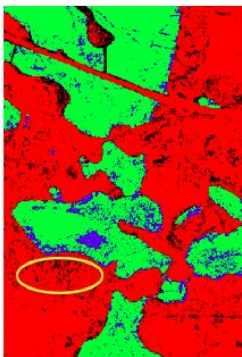
(c) Features RGB 2003



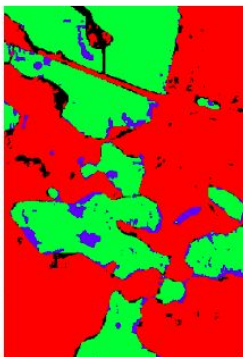
(d) Features RGB,NIR 2005



(e) Features NDVI 2005



(f) Prediction of DT



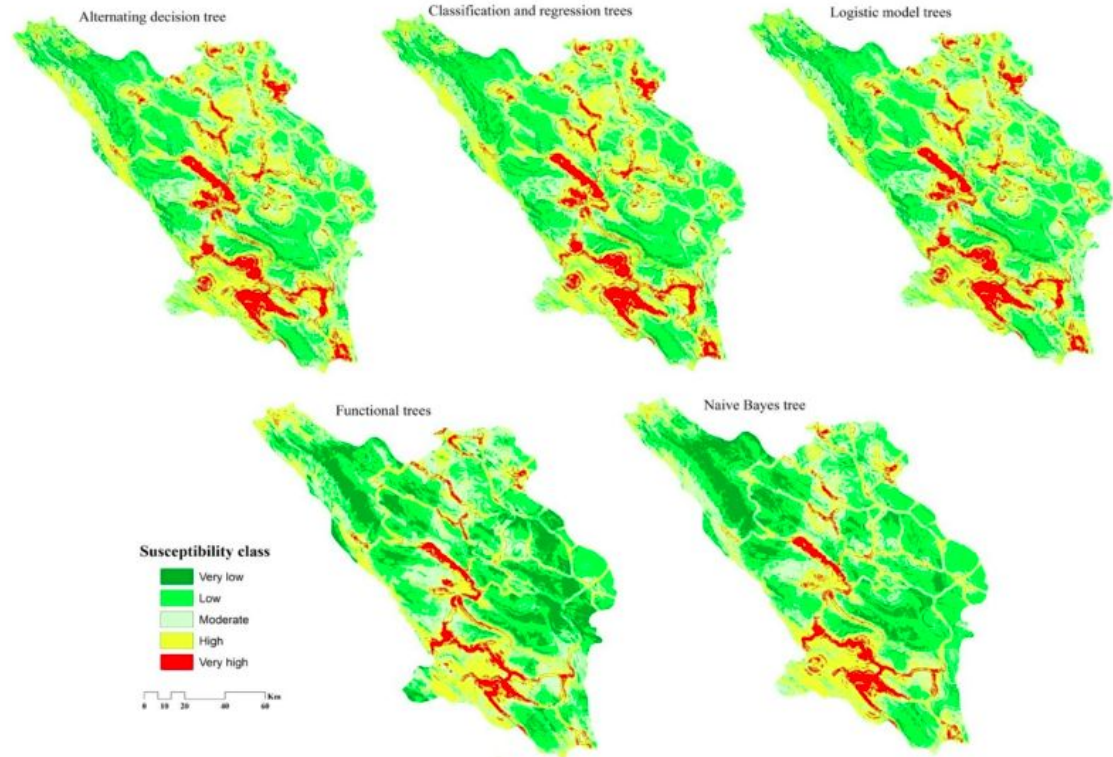
(g) Prediction of FTSDT



(h) Legend of prediction map

Wildfire spatial pattern analysis in the Zagros Mountains, Iran: A comparative study of decision tree based classifiers.

<https://www.sciencedirect.com/science/article/abs/pii/S157495411730167X>



Recordar

Regresión lineal

Regresión

Regresión logística

Clasificación

SVM

Árboles de decisión

Regresión y clasificación

Con árboles de decisión podemos hacer regresiones y clasificaciones, al igual que con SVM

- Introduction to statistical learning. Capítulo 8
- Grokking Machine Learning. Capítulo 9
- The Elements of Statistical Learning. Capítulo 9
- Kuhn, M. & Johnson K. Applied Predictive Modeling.
- Provost, F & Fawcett T. Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking
- Raschka, Sebastian. <https://www.youtube.com/watch?v=z2n8kHXkwtM>
- Raschka, Sebastian. Apuntes de clase
(<https://sebastianraschka.com/blog/2021/ml-course.html#l06---decision-trees>)

Práctica con Python